

CAS4160 Homework 3: Q-Learning

[Your name]
[Your student ID]

1 Introduction

이 과제의 목표는 **Q-learning 방법**을 이해하는 데 도움을 주는 것으로, Deep Q-Network (DQN)과 Double DQN을 포함합니다.

이 과제의 첫 번째 부분은 DQN에 대한 퀴즈를 포함합니다. 그 다음 과제의 두 번째 부분에서는 작동하는 Q-learning 버전을 구현하고, Atari 게임을 플레이하기 위한 Q-learning을 평가하게 됩니다. 우리의 코드는 입력이 저차원의 숫자 리스트인 state-based 환경(Cartpole처럼)뿐만 아니라, 픽셀에서 직접 학습하는 것(BankHeist처럼)도 지원합니다!

이 과제는 GPU에서 실행하는 것이 더 빠르지만, CPU로도 충분히 완료할 수 있습니다. 따라서 GPU를 사용할 수 없는 경우 VESSL AI 또는 Colab을 사용하는 것을 추천합니다. 4.2 절은 GPU 사용 시 약 15분, 사용하지 않을 경우 30분 정도 소요되며, 5.2 절은 GPU 사용 시 약 12시간, 사용하지 않을 경우 총 24시간 정도 소요됩니다.

2 Deep Q-Network Quiz

강의에서 배운 Deep Q-Network (DQN)에 대해 복습해 보겠습니다. 다음 True/False 문제를 풀어 주세요:

I. Q-Learning은 off-policy 샘플을 활용할 수 없기 때문에 샘플 효율성이 낮습니다.

- ☐ 참
- ☐ 거짓

II. 액터가 없으면 연속 행동 공간에서 모든 가능한 행동에 대한 Q-value를 평가하는 것이 불가능합니다.

- ☐ 참
- ☐ 거짓

III. 신경망을 이용한 Q-learning 적용 시 주요 어려움 중 하나는 moving target 문제입니다. 이는 에이전트가 Q-value와 target value를 동일한 신경망으로 추정할 때 발생합니다. 이를 방지하기 위해 inner loop 안에서 fixed target network를 사용할 수 있습니다.

- ☐ 참
- ☐ 거짓

IV. 시간이 지남에 따라 더 많은 탐험을 장려하기 위해 epsilon scheduling을 자주 사용합니다.

- ☐ 참
- ☐ 거짓

3 Code Structure Overview

훈련은 run_hw3.py 스크립트로 시작합니다. 이 스크립트는 run_training_loop 함수를 포함하며, 여기서 훈련, 평가, 그리고 로깅이 이루어집니다.

DQNAgent인 DQN 에이전트를 `cas4160/agents/dqn_agent.py` 파일에서 구현하고, DQN 훈련 루프를 `cas4160/scripts/run_hw3.py`에서 구현하게 됩니다. 또한, 다음 파일들을 처음부터 철저히 읽어보시기 바랍니다:

- `cas4160/env_configs/dqn_basic_config.py`: 기본 DQN 문제(CartPole-v1, LunarLander-v2)를 위한 네트워크를 구축하고 구성을 생성합니다.
- `cas4160/env_configs/dqn_atari_config.py`: Atari DQN 문제(Breakout, BankHeist-v5)를 위한 네트워크를 구축하고 구성을 생성합니다.
- `cas4160/infrastructure/replay_buffer.py`: replay buffer의 구현입니다. DQN에서 frame-stack과 함께 observation을 효율적으로 저장하고 샘플링하기 위해 MemoryEfficientReplayBuffer를 사용합니다. 각 메서드가 어떤 역할을 하는지(특히 프레임 이후에 호출되는 insert와 trajectory의 첫 observation을 삽입하는 on_reset) 이해하고, 일반 replay buffer와 어떻게 다른지 확인해 보세요.
- `cas4160/infrastructure/atari_wrappers.py`: Atari 환경에 특화된 wrapper들을 포함하고 있습니다. 이러한 wrapper들은 어려운 Atari 환경을 제대로 작동하게 하는 데 핵심이 될 수 있습니다!

3.1 Important Implementation Tricks

제공된 코드에는 훈련을 안정화하기 위한 몇 가지 구현 트릭이 포함되어 있습니다. 이 트릭들을 활성화하기 위해 따로 할 일은 없지만, 구현 내용을 자세히 살펴보고 왜 이러한 방법들이 효과적인지 생각해 보시기 바랍니다.

- **ϵ -greedy 액터를 위한 Exploration scheduling.** ϵ 을 처음에는 높은 값(무작위 샘플링에 가까운 값)으로 시작하여 훈련 동안 점차 작은 값으로 감소시킵니다 (exploration_schedule in `cas4160/scripts/run_hw3.py`).
- **Learning rate scheduling.** 학습률을 훈련 초기에 높은 값에서 훈련 끝에는 낮은 값으로 감소시킵니다 (DQNAgent.lr_scheduler).
- **Gradient clipping.** 그라디언트 norm이 threshold보다 크면, norm이 threshold와 같아지도록 그라디언트를 스케일링합니다 (DQNAgent.update_critic).
- **Atari wrappers.** (in `cas4160/infrastructure/atari_wrappers.py`)
 - **Grayscale.** RGB 이미지 ($84 \times 84 \times 3$)를 그레이스케일 이미지 (84×84)로 변환합니다.
 - **Frame-skip.** 동일한 action을 4 스텝 동안 유지하고 중간 입력은 무시합니다.
 - **Frame-stack.** 마지막 4개의 그레이스케일 프레임을 스택하여 입력으로 사용합니다 ($84 \times 84 \times 4$).

4 Deep Q-Learning

4.1 Implementation

기본 DQN 알고리즘을 구현하세요. Q-네트워크와 target network에 대한 업데이트, ϵ -greedy 샘플링, trajectory 수집 함수를 구현하게 됩니다:

- `cas4160/agents/dqn_agent.py`의 update_critic 함수에서 미구현된 부분(TODO(student)로 표시된 부분)을 채워 Q-네트워크 업데이트를 구현하세요.
- `cas4160/agents/dqn_agent.py`의 update 함수를 구현하세요. 이 함수는 update_critic을 호출하고, 필요 시 target critic을 업데이트합니다.
- `cas4160/agents/dqn_agent.py`의 get_action 함수에서 ϵ -greedy 샘플링을 구현하세요.

- cas4160/scripts/run_hw3.py의 TODO 부분을 구현하세요.
- cas4160/infrastructure/utils.py의 sample_trajectory 함수의 TODO 부분을 구현하세요. **Hint:** trajectory는 두 가지 방식으로 종료될 수 있습니다: trajectory의 실제 종료(terminated=True, 보통 catastrophic failure, 예: 충돌로 인해 발생) 또는 truncation(truncated=True), trajectory가 실제로 끝나지는 않았지만 어떤 이유로 시뮬레이션을 중단하는 경우(예: 최대 에피소드 길이 초과)입니다.

gymnasium에서는 env.step의 반환값 중 두 개의 boolean 값이 이에 대응합니다. 여기서 terminated 플래그는 trajectory의 실제 종료를 의미하고, truncated 플래그는 최대 에피소드 길이 도달 등 다른 이유로 인한 trajectory의 truncation을 의미합니다. 후자의 경우에도 환경은 reset해야 하지만, replay buffer에 저장되는 TD-update용 done 플래그는 False여야 합니다.

4.2 Experiment

DQN-CartPole. CartPole-v1 환경에서 experiments/dqn/cartpole.yaml 설정 파일을 사용하여 DQN 구현을 테스트하세요. 약 300K 스텝(약 15분) 만에 reward가 거의 500에 도달해야 합니다.

```
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/cartpole.yaml --seed 1
```

- environment steps를 x -축으로, eval return (eval_return)을 y -축으로 하여 learning curve를 plot하세요. Homework 1 또는 2에서 사용했던 cas4160/scripts/parse_tensorboard.py를 재사용하거나 수정하여 활용할 수 있습니다.

Answer:

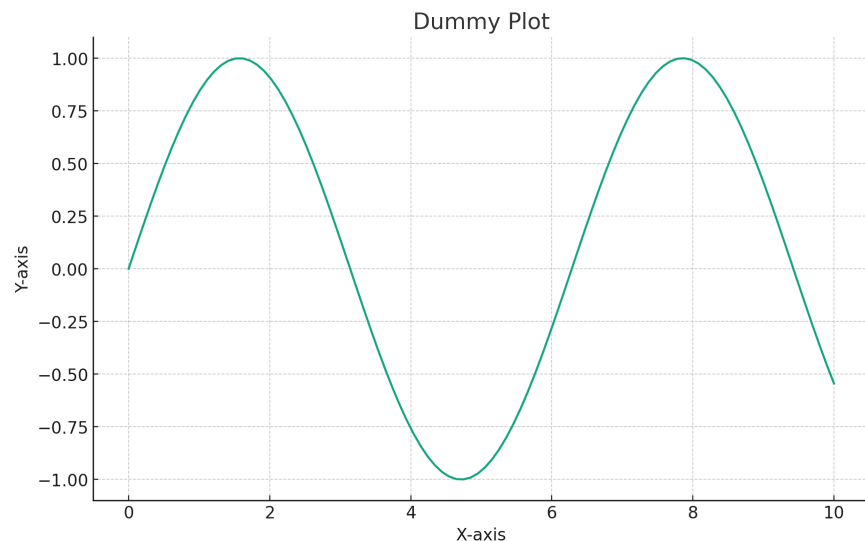


Figure 1: 여기에 본인의 plot을 넣고, 캡션을 추가하세요.

설명을 작성하세요.

- CartPole-v1에서 DQN을 실행하되, experiments/dqn/cartpole.yaml 설정 파일을 수정하여 experiments/dqn/cartpole.yaml 파일을 새로 만드세요 (learning rate를 0.05로 변경, 기본값은 0.001). 이 설정 파일로 실행한 후, (a) 예측된 Q -values, (b) critic error, (c) eval returns에 어떤 변화가 발생하는지 설명하세요. 두 가지 learning rate에 대한 결과를 비교하는 세 개의 plot을 제공하세요. 수업에서 배운 어떤 내용과 관련이 있는지 설명할 수 있나요? 이유와 설명을 작성하세요. **Answer:**

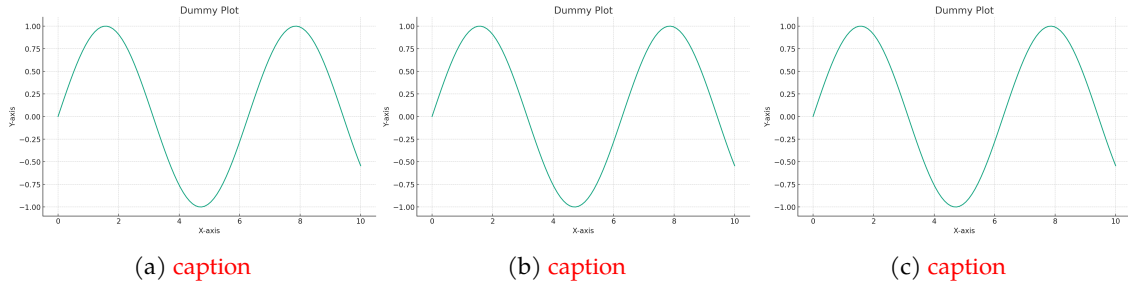


Figure 2: 여기에 본인의 plot을 넣고, 캡션을 추가하세요.

설명을 작성하세요.

5 Double Q-Learning

5.1 Implementation

학습을 안정화시켜 보겠습니다. double-Q trick은 critic 업데이트에서 overestimation bias를 피하기 위해, next action $\mathbf{a}'$ 을 선택하는 것과 그 값을 추정하는 것에 서로 다른 두 개의 네트워크를 사용합니다:

$$\mathbf{a}' = \arg \max_{\mathbf{a}'} Q_{\phi}(\mathbf{s}', \mathbf{a}')$$

$$Q_{\text{target}} = r + \gamma(1 - d_t)Q_{\phi'}(\mathbf{s}', \mathbf{a}').$$

우리의 경우, action의 값을 추정하는 데는 target network $Q_{\phi'}$ 를 계속 사용하되, action 선택은 Q_{ϕ} (online Q-network)를 사용합니다. 이 기능을 cas4160/agents/dqn_agent.py에서 구현하세요.

5.2 Experiments

이번 문제는 GPU 사용 시 각 실험이 약 2시간 (총 12시간), GPU 미사용 시 **4시간 (24시간 총 소요)** 정도 걸리니, 미리 시작하시기 바랍니다!

- BankHeist-v5 환경에서 3가지 다른 seed로 DQN을 실행하세요:

```
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist.yaml --seed 1
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist.yaml --seed 2
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist.yaml --seed 3
```

return은 약 150까지 향상되어야 합니다.

- BankHeist-v5 환경에서 3가지 seed로 Double DQN을 실행하세요:

```
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist_ddqn.yaml --seed 1
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist_ddqn.yaml --seed 2
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist_ddqn.yaml --seed 3
```

Double DQN의 3가지 seed에 대한 return을 **빨간색**으로, “vanilla” DQN 결과를 **파란색**으로 같은 축에 plot하세요. DQN과 Double DQN을 비교하고, 이 차이를 무엇이 설명할 수 있는지 자신의 말로 서술하세요. **Note:** 결과는 예상과 다를 수 있습니다. 경우에 따라 DQN이 Double DQN과 비슷하거나 오히려 더

좋은 성능을 보일 수도 있습니다. 이는 제한된 훈련 시간, 환경 특성, 또는 다른 seed로 인한 randomness 등의 실질적인 요인 때문일 수 있습니다. 이 부분은 실험 결과 자체가 아니라 구현의 정확성을 중심으로 평가될 것입니다. **Answer:**

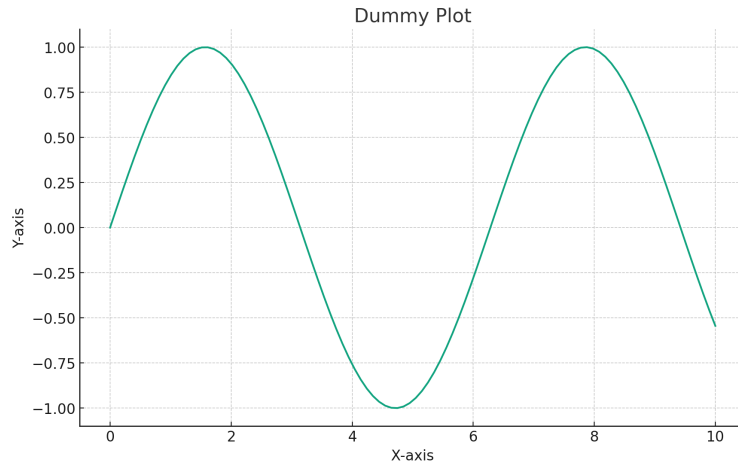


Figure 3: 여기에 본인의 plot을 넣고, 캡션을 추가하세요.

설명을 작성하세요.

6 Experimenting with Hyperparameters

CartPole-v1 환경에서 Q-learning의 하이퍼파라미터에 대한 민감도를 분석해 보겠습니다. 본인이 선택한 하나의 하이퍼파라미터를 정하고, 기본값 외에 해당 하이퍼파라미터의 최소 3가지 다른 설정으로 실험을 수행한 후, 총 4가지 설정에 대한 eval return을 모두 같은 그래프에 plot하세요. 캡션에서 왜 이 하이퍼파라미터를 선택했는지 설명하세요. experiments/dqn/hyperparameters 폴더에 4개의 config 파일을 생성하세요(cas4160/env_configs/basic_dqn_config를 참고하여 변경 가능한 하이퍼파라미터를 확인하세요). 어떤 base YAML 파일이든 참조하여 사용할 수 있습니다.

Hyperparameter 옵션 예시는 다음과 같습니다:

- Learning rate
- Network architecture
- Exploration schedule (또는 원한다면 ϵ -greedy 대신 다른 대안을 직접 구현해도 됩니다)

Answer:

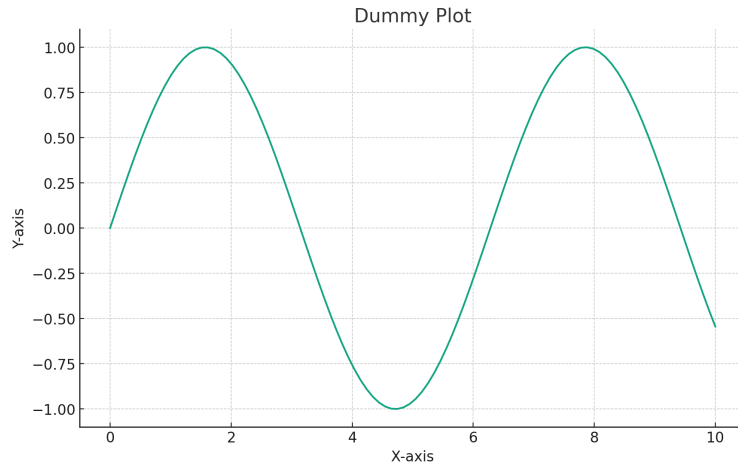


Figure 4: 여기에 본인의 plot을 넣고, 캡션을 추가하세요.

설명을 작성하세요.

7 Submission

Please submit the code and the “**report**” in a single zip file, `hw3_[YourStudentID].zip`. Do not include videos and logs as the file size should be less than 20MB. The structure of the submission file should be:

```
hw3_[YourStudentID].zip
├── hw3_[YourStudentID].pdf
├── cas4160/
│   └── ...codes
└── ...
```